

Lab 5 — Coverage

FIFO

Task 1

Question 1.1: Covergroup Bins

The testbench defines one covergroup (`fifo_cov`) and one `cover` property directive. In total there are **13 covergroup bins + 1 cover directive**.

Coverpoint `full` (1 bin)

1. `went_full` — bin for `full == 1`

Coverpoint `empty` (1 bin)

2. `went_empty` — bin for `empty == 1`

Coverpoint `dut.count` (3 bins)

3. `empty_bin` — occupancy equals 0
4. `mid_bins` — occupancy in range `[1 : DEPTH-1]`
5. `full_bin` — occupancy equals `DEPTH`

Coverpoint `wr_en` (2 auto-bins)

6. `auto[0]` — `wr_en == 0`
7. `auto[1]` — `wr_en == 1`

Coverpoint `rd_en` (2 auto-bins)

8. `auto[0]` — `rd_en == 0`
9. `auto[1]` — `rd_en == 1`

Cross `wr_en` × `rd_en` (4 bins)

10. `write_only` — `wr_en=1, rd_en=0`
11. `read_only` — `wr_en=0, rd_en=1`
12. `both_off` — `wr_en=0, rd_en=0`
13. `illegal` — `wr_en=1, rd_en=1`

Cover Directive (+1)

- D1. `cover property (becomes_empty_after_read)` — covers the sequence where the FIFO transitions to empty after a read when `dut.count > 0`.

Question 1.2: Bins Covered After Reset Only

The following **5 bins** are covered just from reset:

1. `went_empty` — after reset the FIFO is empty, so `empty == 1`
2. `empty_bin` — `dut.count == 0` after reset
3. `wr_en auto[0]` — `wr_en` stays 0
4. `rd_en auto[0]` — `rd_en` stays 0
5. `both_off` — cross bin where `wr_en=0`, `rd_en=0`

Task 2

Question 2.1: Additionally Covered Bins

After enabling the provided stimulus, the following **5 additional bins** are now covered:

1. `write_only` — cross bin where `wr_en=1`, `rd_en=0`
2. `read_only` — cross bin where `wr_en=0`, `rd_en=1`
3. `wr_en auto[1]` — `wr_en == 1`
4. `rd_en auto[1]` — `rd_en == 1`
5. `mid_bins` — occupancy in range `[1 : DEPTH-1]`

Task 3

Missing Bins After Task 2

The following **3 bins + 1 cover directive** remained uncovered:

1. `went_full` — `full == 1`
 2. `full_bin` — `dut.count == DEPTH`
 3. `illegal` — cross bin where `wr_en=1`, `rd_en=1`
- D1. `becomes_empty_after_read` — FIFO transitions to empty after a read when `dut.count > 0`

Additional Stimulus

Fully fill and fully empty the FIFO. Increased the number of consecutive writes to exceed DEPTH (16), driving the FIFO to full occupancy, then increased the number of consecutive reads to drain it completely back to empty.

Hits bins:

- `went_full` — FIFO reaches full capacity
- `full_bin` — `dut.count == DEPTH`
- `becomes_empty_after_read (D1)` — FIFO drains to empty after a read while `dut.count > 0`

Drive both `wr_en` and `rd_en` on the same cycle. Added stimulus that asserts `wr_en=1` and `rd_en=1` simultaneously.

Hits bin:

- `illegal` — cross bin where `wr_en=1`, `rd_en=1`

Cache

Task 4

Covergroup Bins

The testbench defines one covergroup (`cache_cov`) with **9 coverpoints/crosses** totalling **27 bins**, plus **7 cover property** directives.

Coverpoint write (2 auto-bins)

1. `auto[0]` — `write == 0`
2. `auto[1]` — `write == 1`

Coverpoint read (2 auto-bins)

3. `auto[0]` — `read == 0`
4. `auto[1]` — `read == 1`

Coverpoint hit (2 auto-bins)

5. `auto[0]` — `hit == 0`
6. `auto[1]` — `hit == 1`

Cross write × read (4 bins)

7. `write_only` — `write=1, read=0`
8. `read_only` — `write=0, read=1`
9. `both_off` — `write=0, read=0`
10. `read_over_write` — `write=1, read=1`

Cross write $\times$ hit (4 bins)

- 11. `write_hit` — write=1, hit=1
- 12. `write_miss` — write=1, hit=0
- 13. `auto[write=0][hit=1]` — not writing, hit asserted (carried from prior op)
- 14. `auto[write=0][hit=0]` — not writing, no hit

Cross read $\times$ hit (4 bins)

- 15. `read_hit` — read=1, hit=1
- 16. `read_miss` — read=1, hit=0
- 17. `auto[read=0][hit=1]` — not reading, hit asserted (carried from prior op)
- 18. `auto[read=0][hit=0]` — not reading, no hit

Coverpoint `dut.tag` (3 bins)

- 19. `lower_bins` — tag in range [0 : TAG_LO]
- 20. `mid_bins` — tag in range [TAG_LO+1 : TAG_HI-1]
- 21. `high_bins` — tag in range [TAG_HI : TAG_MAX]

Coverpoint `dut.index` (3 bins)

- 22. `lower_bins` — index in range [0 : INDEX_LO]
- 23. `mid_bins` — index in range [INDEX_LO+1 : INDEX_HI-1]
- 24. `high_bins` — index in range [INDEX_HI : INDEX_MAX]

Coverpoint `dut.offset` (3 bins)

- 25. `lower_bins` — offset in range [0 : OFFSET_LO]
- 26. `mid_bins` — offset in range [OFFSET_LO+1 : OFFSET_HI-1]
- 27. `high_bins` — offset in range [OFFSET_HI : OFFSET_MAX]

Cover Directives (7)

- D1. `p_read_replace` — read to a valid line with tag mismatch (conflict miss / replacement)
- D2. `p_write_replace` — write to a valid line with tag mismatch (conflict miss / replacement)
- D3. `p_read_cold_miss` — read to an invalid line (compulsory miss)
- D4. `p_write_cold_miss` — write to an invalid line (compulsory miss)
- D5. `p_write_then_read_hit` — write followed by read hit to the same address within 5 cycles (data round-trip)

- D6. `p_read_miss_then_hit` — read miss followed by read hit to the same address within 5 cycles (cache fill path)
- D7. `p_hit_deasserts` — hit transitions $1 \rightarrow 0$ (verifies hit is not stuck high)

Bins Covered by Initial Random Test

26 of 27 covergroup bins and 5 of 7 cover directives were hit. The following 3 were not:

- `read_over_write` (cross `write` $\times$ `read`)
- `p_write_then_read_hit`
- `p_read_miss_then_hit`

Task 5

Stimulus changes made to achieve 100

1. Weighted constrained-random operation class. Replaced the 50/50 `$urandom_range` with a SystemVerilog transaction class using a `dist` constraint, enabling all four operation combinations to be generated:

Operation	Weight	Signals
Read only	45%	<code>read=1, write=0</code>
Write only	45%	<code>read=0, write=1</code>
Simultaneous	5%	<code>read=1, write=1</code>
Idle	5%	<code>read=0, write=0</code>

Covers: `read_over_write` — the 5% simultaneous weight guarantees `read=1, write=1` is generated within 200 cycles.

2. Directed sequences for same-address access. Two directed loops were added after the random phase:

- *Write then read-hit:* writes to a random address, then reads the same address on the next cycle. Repeated 20 times. Covers `p_write_then_read_hit`.
- *Read-miss then read-hit:* reads a random address (miss), then reads the same address on the next cycle (guaranteed hit). Repeated 20 times. Covers `p_read_miss_then_hit`.